

Session 6 Homework Walkthrough

This homework continues the same B4F Hub project. We finished extracting the server routes, added an endpoint for reporting an opportunity, and connected that endpoint to the opportunity detail page.

We followed the project's existing syntax: plain Express controller functions, routers, shared arrays, `fetch`, and React state. We completed CORE, STRETCH, and CHALLENGE functionality. We skipped the requested CSS work because we chose to focus on the logic and leave styles unchanged.

1. What changed?

Part	Before	After
Health endpoint	Its handler was inline in <code>server/index.js</code> .	Its handler lives in a controller, with a separate router.
Server entry point	It included health response logic and direct array reads.	It sets up middleware, mounts three routers, and starts the server.
Reporting opportunities	There was no endpoint for reports.	POST <code>/api/opportunities/:id/report</code> validates and stores reports in memory.
Frontend API	It could load and apply to opportunities.	It can also submit a report and read the server's error message.
Opportunity detail page	It had an Apply button.	It also has a reason input and Report button with notifications.

The initial working files also contained merge conflicts in `server/index.js` and `server/store.js`. We resolved them while preserving both the posts and opportunities routers and their shared data. We also fixed an imported-array reassignment in post creation, explained below.

2. CORE: extracting the health endpoint

Before: the handler was inside `index.js`

The health route's original logic was:

```
app.get("/api/health", (req, res) => {
  res.json({
    status: "ok",
    opportunities: opportunities.length,
    posts: posts.length,
  });
});
```

This made `index.js` responsible for both starting the application and building the health response.

After: controller, router, and mount

We created `server/controllers/health.js`:

```
import { opportunities, posts } from "../store.js";

export function getHealth(req, res) {
  res.json({
    status: "ok",
    opportunities: opportunities.length,
    posts: posts.length,
  });
}
```

The response stays the same. A controller is just a normal JavaScript function that receives `req` and `res`. Moving it makes the server easier to read without changing the endpoint's behavior.

We created `server/routes/health.js`:

```
import express from "express";
import { getHealth } from "../controllers/health.js";

const router = express.Router();

router.get("/", getHealth);

export default router;
```

Then we imported and mounted it in `server/index.js`:

```
import healthRouter from "../routes/health.js";

app.use("/api/health", healthRouter);
```

The mount path `/api/health` and the router path `/` together serve `GET /api/health`. `express.Router()` groups routes; it does not create another server. We still have one `app.listen()`.

`index.js` now mounts `health`, `posts`, and `opportunities` routers. It no longer imports or reads the `posts` or `opportunities` arrays. The health controller reads their current lengths from `store.js`, so creating a post changes the count returned by the next health request.

3. STRETCH: adding the report endpoint

We added `reportOpportunity(req, res)` and a module-level array to `server/controllers/opportunities.js`:

```
const reports = [];
```

Because this array is outside the handler, it survives between requests in the same server process. It resets when the server restarts. The homework specifically keeps reports here; the existing shared posts and opportunities remain in `store.js`.

Reading the request

```
const id = Number(req.params.id);
const { reason } = req.body ?? {};
const opportunity = opportunities.find((candidate) => candidate.id === id);
```

For `/api/opportunities/101/report`, `req.params.id` is the string `"101"`. `Number()` converts it so it can match the numeric IDs in our data.

`reason` comes from the JSON body. The existing `app.use(express.json())` parses incoming JSON before the request reaches the router. `req.body ?? {}` provides an empty object if the body is missing or null, so destructuring does not throw.

Checking existence before content

```
if (!opportunity) {
  return res
    .status(404)
    .json({ error: `No opportunity found with id ${id}.` });
}

if (typeof reason !== "string" || reason.trim().length < 3) {
  return res.status(400).json({
    error: "reason is required and must be a string of at least 3 characters after trimming.",
  });
}
```

The order matters. If the opportunity does not exist, we return 404, even when the reason is also invalid. If the opportunity exists but the reason is invalid, we return 400.

The `typeof` check comes before `.trim()`. JavaScript's `||` short-circuiting means `.trim()` is only called when `reason` is a string. Trimming also means spaces do not count toward the minimum length.

Each `return` stops the handler immediately. Invalid requests cannot reach the code that stores a report or sends the success response.

Creating the report

```
const report = {
  id: reports.length + 1,
  opportunityId: id,
  reason: reason.trim(),
  createdAt: new Date().toISOString(),
};

reports.push(report);

res.status(201).json(report);
```

The report has its own ID, the related opportunity ID, the trimmed reason, and a timestamp. Using `reports.length + 1` works for this small array because reports are only added; there is no deletion endpoint. 201 means a new resource was created.

We wired the controller into `server/routes/opportunities.js`:

```
router.post("/:id/report", reportOpportunity);
```

That router is already mounted at `/api/opportunities`, so the full endpoint is `POST /api/opportunities/:id/report`.

4. CHALLENGE: connecting React to the server

The API function

In `client/src/api.ts`, we added the response type:

```
export interface OpportunityReport {
  id: number;
  opportunityId: number;
  reason: string;
  createdAt: string;
}
```

Then we added the same request pattern used by the existing API functions:

```
export async function reportOpportunity(
  id: number,
  reason: string,
): Promise<OpportunityReport> {
  const response = await fetch(`${OPPORTUNITIES_URL}/${id}/report`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ reason }),
  });

  if (!response.ok) {
    throw new Error(
      await readErrorMessage(response, "Could not submit your report."),
    );
  }

  return (await response.json()) as OpportunityReport;
}
```

`JSON.stringify()` converts the `reason` object into the JSON body. The header tells Express what format we sent. We check `response.ok` because `fetch` does not automatically throw for HTTP responses such as 400 and 404.

The existing `readErrorMessage()` helper preserves the backend's validation message. The TypeScript cast describes the expected response to the compiler; it does not validate the response at runtime.

The frontend and backend functions share the name `reportOpportunity`, but they have different jobs: the frontend function sends an HTTP request, and the backend controller handles it.

Reusing the `handleApply` pattern

In `client/src/pages/OpportunityDetailPage.tsx`, we added:

```
const [reason, setReason] = useState("");
const [isReporting, setIsReporting] = useState(false);
```

`reason` holds the input value. `isReporting` tracks whether a request is in progress. These values only belong to this form, so ordinary component state is enough.

The new handler follows the same `try / catch / finally` structure as `handleApply`:

```

async function handleReport() {
  if (!opportunity || !reason.trim() || isReporting) return;

  setIsReporting(true);

  try {
    await reportOpportunity(opportunity.id, reason);
    setReason("");
    notify("Your report was submitted.", "success");
  } catch (error) {
    const message =
      error instanceof Error
        ? error.message
        : "Could not submit your report.";
    notify(message, "error");
  } finally {
    setIsReporting(false);
  }
}

```

On success, we clear the input and use the existing `useNotify()` system to show confirmation. On failure, we show the error and keep the reason so the user can correct it. `finally` resets the loading state in both cases.

The input and button

We placed a labeled input and a Report button beneath Apply. The input is controlled with `value={reason}` and `onChange={(event) => setReason(event.target.value)}`.

The button uses:

```

onClick={handleReport}
disabled={!reason.trim() || isReporting}

```

It is disabled when the input is empty or contains only spaces, and while a request is in progress. The input is also disabled during submission. The button text changes to `Reporting...` while waiting.

The frontend blocks empty submissions. The backend still enforces the three-character minimum, including when someone calls the endpoint directly. A one-character reason therefore reaches the server and produces a validation notification.

5. Fixing shared post state during the extraction

The extracted post controller originally contained:

```
posts = [newPost, ...posts];
```

This tries to reassign an imported binding. JavaScript modules do not allow a file to reassign a variable imported from another module.

We changed it to:

```
posts.unshift(newPost);
```

`unshift()` adds the post to the beginning of the existing array. The controller and health endpoint still read the same array exported by `store.js`.

This applies Session 6's shared-state lesson: `const` prevents reassignment of the array variable, but the contents of the array can still change. The existing `claimNextPostId()` function remains responsible for updating the post ID counter inside the module that owns it.

6. The complete request flow

```
User types a reason and clicks Report
-> handleReport() sets isReporting to true
-> api.ts sends POST /api/opportunities/101/report
-> express.json() parses the JSON body
-> the existing logging middleware logs the request
-> the opportunities router selects reportOpportunity
-> the controller checks the opportunity, then the reason
-> a valid report is stored and returned with status 201
-> the API function returns the parsed report
-> React clears the input and shows the success notification
-> finally resets isReporting
```

For a 400 or 404 response, the API function throws an error. The component's `catch` displays that error instead of clearing the input.

7. How this applies Session 6

Session 6 concept	Where we used it	Why it matters
Plain controller functions	<code>getHealth</code> and the backend <code>reportOpportunity</code>	Request handling has a clear home.
<code>express.Router()</code>	New health router and existing opportunities router	Routes are grouped by responsibility.
<code>app.use(mountPath, router)</code>	Three router mounts in <code>index.js</code>	Mount paths combine with relative router paths.
<code>req.params</code> and <code>req.body</code>	Opportunity ID and report reason	URL values and body values are read separately.
<code>req.body ?? {}</code>	Report controller	Missing bodies reach validation safely.
Plain validation and early returns	Existence and reason checks	Invalid requests stop before modifying data.
HTTP status codes	200 for health, 400 for invalid reasons, 404 for missing opportunities, 201 for created reports	Clients can distinguish outcomes.
Shared mutable state	Posts and opportunities in <code>store.js</code>	Controllers see the same current data.
Existing React request pattern	<code>handleReport</code> , modeled on <code>handleApply</code>	Loading, success, and failure follow familiar code.

The homework uses no database, authentication, validation library, new npm dependency, custom Express error middleware, or Redux state for reporting. The already-declared `dotenv` dependency was installed locally so the server could run; the dependency list did not change.

8. What we verified

These checks were completed when implementing the homework:

- Health returns the real opportunity and post counts, including the updated post count after creating a post.
- A valid report returns 201, a trimmed reason, its opportunity ID, and a timestamp. Consecutive reports receive different IDs.
- Missing, non-string, whitespace-only, and too-short reasons return 400 for an existing opportunity.
- A nonexistent opportunity returns 404, including when the reason is invalid.
- Existing post creation, liking, opportunity type filtering, applying, and duplicate-application handling still work.
- In the browser, a successful report displayed the notification, cleared the input, and disabled Report again. The server logged the actual POST request.
- `npm run build` in `client/` passed.
- `npm run lint` in `client/` finished with zero errors and two existing warnings: the notification context's mixed exports and the detail page effect's missing load dependency.
- `node --check server/index.js` passed from the repository root.

The changes are local. No commit or push was made as part of this homework implementation.